TP3 — Stockage Proxmox

iSCSI · NFS · LVM-thin · ZFS · Résilience

Dimitri ANGELY — M2 Infrastructure Cloud — Juin 2026

Environnement de travail

Composant	Détail
Nœud Proxmox	1 disque sda 200G — pve-root 59.7G — pve-data LVM-thin 112.9G
Disques ZFS	3 fichiers loopback 10G chacun (/var/lib/zfs-disks/)
VM 110 srv-nfs-iscsi	192.168.131.147 — serveur NFS + cible iSCSI
VM 120 zfs-test01	192.168.131.148 — nginx sur ZFS (disque déplacé)
VM 121 restaurée	192.168.131.149 — restauration depuis backup NFS

Partie 1 — Analyse des stockages existants

> **Pourquoi**
> Cartographier l'existant avant toute modification. Un stockage implicitement utilisé par une VM ne doit pas être touché sans comprendre les dépendances.

Commande

```
pvesm status
```

Sortie

```
Name             Type     Status      Total (KiB)      Used (KiB) Available (KiB)
%
local             dir     active        61349692         6266948        51933944
10.22%
local-lvm     lvmthin     active       118427648        20298498        98129149
17.14%
```

> **Analyse**
> local (Directory) = stockage fichier sur /var/lib/vz. Héberge ISO, backups, templates. local-lvm (LVM-thin) = stockage bloc pour les disques VM. Thin pool pve/data à 17% — allocation dynamique active.

Partie 2 — Stockage NFS

> **Pourquoi NFS**
> NFS exporte un répertoire accessible en réseau comme un dossier monté. Contrairement à iSCSI, NFS ne présente pas de bloc brut — c'est du stockage fichier partagé. Idéal pour les backups et ISO partagés dans un cluster.

Configuration du serveur NFS (VM 110 — 192.168.131.147)

```
apt update && apt install -y nfs-kernel-server
mkdir -p /srv/nfs/proxmox && chmod 777 /srv/nfs/proxmox
echo "/srv/nfs/proxmox
192.168.131.0/24(rw,sync,no_subtree_check,no_root_squash)" >> /etc/exports
exportfs -ra && systemctl enable --now nfs-server
exportfs -v
```

Sortie exportfs -v

```
/srv/nfs/proxmox

192.168.131.0/24(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,no_root_squash,no_all_
squash)
```

Ajout dans Proxmox

```
showmount -e 192.168.131.147
pvesm add nfs nfs-backup \
  --server 192.168.131.147 \
  --export /srv/nfs/proxmox \
  --content backup,iso
pvesm status | grep nfs
```

Résultat

```
Export list for 192.168.131.147:
/srv/nfs/proxmox 192.168.131.0/24

nfs-backup   nfs   active   7669824   1428672   5830080   18.63%
```

Partie 3 — Stockage iSCSI

> **NFS vs iSCSI**
> iSCSI expose un périphérique bloc (LUN/disque virtuel) que le client partitionne et formate selon ses besoins. NFS exporte un répertoire. C'est la distinction fondamentale fichier vs bloc.

Configuration de la cible iSCSI (VM 110)

```
apt install -y targetcli-fb
dd if=/dev/zero of=/srv/iscsi/lun0.img bs=1M count=3072 status=progress
targetcli
  /> /backstores/fileio create lun0 /srv/iscsi/lun0.img 3G
  /> /iscsi create iqn.2026-06.local.proxmox:storage
  /> /iscsi/iqn.2026-06.local.proxmox:storage/tpg1/luns create
/backstores/fileio/lun0
  /> /iscsi/iqn.2026-06.local.proxmox:storage/tpg1/ set attribute
authentication=0
  /> /iscsi/iqn.2026-06.local.proxmox:storage/tpg1/ set attribute
generate_node_acls=1
  /> saveconfig && exit
```

Connexion depuis Proxmox

```
iscsiadm -m discovery -t st -p 192.168.131.147
pvesm add iscsi iscsi-store \
  --portal 192.168.131.147 \
  --target iqn.2026-06.local.proxmox:storage \
  --content none
pvesm list iscsi-store
```

Résultat

```
192.168.131.147:3260,1 iqn.2026-06.local.proxmox:storage

Login to [iface: default, target: iqn.2026-06.local.proxmox:storage, portal:
192.168.131.147,3260] successful.

Volid                                                Format  Type            Size
VMID
iscsi-store:0.0.0.scsi-36001405b1ca6228095d46d78240548e6 raw     images    3221225472
```

> **Observation clé**
> Format raw, Type images, aucun MOUNTPOINT. C'est du bloc brut — pas de système de fichiers, pas de dossier. Le client décide ce qu'il en fait (LVM, ext4, ZFS...).

Partie 4 — LVM-thin

Pourquoi LVM-thin pour les VMs

LVM-thin alloue l'espace dynamiquement. Une VM annonce 50 Go mais ne consomme que l'espace réellement écrit. Risque principal : saturation silencieuse du pool si Data% > 80%.

```
lvs pve/data
```

```
  LV   VG  Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  data pve twi-aotz-- 112.94g             23.26  2.08
```

Lecture

twi-aotz-- = thin pool actif. Data% = 23.26% — espace réellement consommé sur 112.94 Go provisionnés. Surveiller ce chiffre en production, seuil d'alerte recommandé : 80%.

Partie 5 — Pool ZFS (Mirror + Simple)

> **Contexte**
> Aucun disque physique libre sur le nœud. Technique loopback : 3 fichiers de 10 Go exposés comme périphériques blocs via losetup. Comportement identique à des disques physiques pour ZFS.

Création des loopbacks

```
mkdir -p /var/lib/zfs-disks
dd if=/dev/zero of=/var/lib/zfs-disks/disk0.img bs=1M count=10240
status=progress
dd if=/dev/zero of=/var/lib/zfs-disks/disk1.img bs=1M count=10240
status=progress
dd if=/dev/zero of=/var/lib/zfs-disks/disk2.img bs=1M count=10240
status=progress
losetup /dev/loop0 /var/lib/zfs-disks/disk0.img
losetup /dev/loop1 /var/lib/zfs-disks/disk1.img
losetup /dev/loop2 /var/lib/zfs-disks/disk2.img
losetup -l
```

Création des pools

```
# Pool simple — 1 disque, aucune redondance (démo uniquement)
zpool create demo-simple /dev/loop0

# Pool mirror — 2 disques, tolérance 1 panne (pool principal)
zpool create tank mirror /dev/loop1 /dev/loop2

zpool list
zpool status
```

```
NAME          SIZE  ALLOC   FREE  CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP    HEALTH
ALTROOT
demo-simple  9.50G   123K  9.50G        -         -     0%     0%  1.00x    ONLINE  -
tank         9.50G  1.51G  7.99G        -         -     1%    15%  1.00x    ONLINE  -

  pool: tank
 state: ONLINE
  scan: scrub repaired 0B in 00:00:02 with 0 errors on Tue Jun  9 21:10:29 2026
config:
        NAME        STATE     READ WRITE CKSUM
        tank        ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            loop1   ONLINE       0     0     0
            loop2   ONLINE       0     0     0
errors: No known data errors
```

Persistance au reboot — service systemd

```
cat > /etc/systemd/system/zfs-loopback.service << 'EOF'
[Unit]
Description=ZFS loopback devices
Before=zfs-import.target
DefaultDependencies=no
[Service]
```

```
Type=oneshot
ExecStart=/sbin/losetup /dev/loop0 /var/lib/zfs-disks/disk0.img
ExecStart=/sbin/losetup /dev/loop1 /var/lib/zfs-disks/disk1.img
ExecStart=/sbin/losetup /dev/loop2 /var/lib/zfs-disks/disk2.img
RemainAfterExit=yes
[Install]
WantedBy=zfs-import.target
EOF
systemctl enable zfs-loopback.service
```

Partie 6 — Datasets ZFS

Dataset vs ZVOL

Un dataset ZFS est un système de fichiers monté à un chemin (stockage fichier). Un ZVOL est un volume bloc sans point de montage (stockage bloc). La colonne MOUNTPOINT dans zfs list est la preuve visuelle.

```
zfs create tank/backups
zfs create tank/iso
zfs set compression=lz4 tank
zfs get compression tank tank/backups tank/iso
zfs list
```

```
NAME          PROPERTY     VALUE           SOURCE
tank          compression  lz4             local
tank/backups  compression  lz4             inherited from tank
tank/iso      compression  lz4             inherited from tank

NAME           USED  AVAIL  REFER  MOUNTPOINT
tank           206K  9.20G    25K  /tank
tank/backups    24K  9.20G    24K  /tank/backups
tank/iso        24K  9.20G    24K  /tank/iso
```

Héritage ZFS

Un seul zfs set compression=lz4 tank propage la propriété à tous les datasets enfants (inherited from tank). Pas besoin de configurer chaque dataset individuellement.

Partie 7 — ZVOL et VM zfs-test01

Création de la VM avec disque ZFS sparse

> **Sparse ZVOL**
> ZFS refuse de créer un ZVOL de 20 Go sur un pool de 9.5 Go sans sparse. La propriété -s (refreservation=none) crée un volume logique de 20 Go sans réserver l'espace physiquement — même principe que LVM-thin.

```
# Créer le ZVOL sparse manuellement
zfs create -V 20G -s tank/vm-120-disk-0
zfs get volsize,refreservation,referenced tank/vm-120-disk-0

# Cloner le template et déplacer le disque vers ZFS (Partie 12)
qm clone 9000 120 --name zfs-test01 --full 0
qm move-disk 120 scsi0 zfs-tank --delete 1
```

```
NAME                PROPERTY        VALUE      SOURCE
tank/vm-120-disk-0  volsize         20G        local
tank/vm-120-disk-0  refreservation  none       default
tank/vm-120-disk-0  referenced      12K        -
```

Preuve visuelle Dataset vs ZVOL

```
zfs list | grep -E 'tank/backups|vm-120'
```

```
tank/backups          24K  1.08G    24K  /tank/backups   <- dataset, MONTÉ
tank/vm-120-disk-0  8.13G  7.65G  1.55G  -               <- ZVOL, MOUNTPOINT = "-"
```

Partie 8 — Compression ZFS lz4

```
zfs set compression=lz4 tank
zfs get compression,compressratio tank
```

```
NAME  PROPERTY       VALUE           SOURCE
tank  compression    lz4             local
tank  compressratio  1.61x           -
```

Résultat

Ratio 1.61x = 61% d'espace économisé sur les données réelles du pool. lz4 est recommandé pour son faible coût CPU. La compression s'applique uniquement aux nouvelles écritures.

Partie 9 — Scrub ZFS

Pourquoi le scrub

ZFS stocke un checksum pour chaque bloc au moment de l'écriture. Le scrub relit tous les blocs et vérifie ces checksums. Il détecte les corruptions silencieuses (bit flip disque) que ext4 ou XFS n'auraient pas vues. Si le pool a de la redondance, correction automatique.

```
zpool scrub tank
zpool status tank | grep scan
```

```
  scan: scrub repaired 0B in 00:00:02 with 0 errors on Tue Jun  9 21:10:29 2026
```

Partie 10 — Mirror vs RAIDZ

Type	Disques min	Tolérance panne	Espace utile	Usage
Pool simple	1	Aucune	100%	Lab uniquement
Mirror	2	1 disque	50%	VMs critiques
RAIDZ1	3	1 disque	~67%	Stockage général
RAIDZ2	4	2 disques	~75%	Haute résilience

```
zpool list
```

```
NAME          SIZE  ALLOC   FREE  CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP    HEALTH  ALTROOT
demo-simple  9.50G   123K  9.50G        -         -     0%     0%  1.00x    ONLINE  -
tank         9.50G  1.51G  7.99G        -         -     1%    15%  1.00x    ONLINE  -
```

Observation

demo-simple : 1 disque, 0% utilisé, AUCUNE redondance. tank : mirror-0 avec loop1+loop2, tolérance 1 panne. La capacité utile du mirror = taille d'un seul des deux disques.

Partie 11 — VM storage-test01 (zfs-test01)

```
# Dans la VM 120 (zfs-test01)
apt install -y nginx curl
echo "zfs-test01 operationnel" > /var/www/html/index.html
systemctl enable --now nginx
curl http://localhost
```

```
zfs-test01 operationnel
```

```
# Depuis le nœud Proxmox
curl http://192.168.131.148
qm config 120 | grep scsi0
```

```
zfs-test01 operationnel

scsi0: zfs-tank:vm-120-disk-0,iothread=1,size=8G
```

Partie 12 — Déplacement de disque

> **Move Disk vs Restauration**
> Le déplacement migre les données d'un stockage vers un autre sans recréer la VM. L'UUID, la config, les snapshots restent intacts. La restauration depuis backup crée une nouvelle VM distincte.

```
# Disque source : local-lvm (après clone du template)
qm config 120 | grep scsi0
# scsi0: local-lvm:vm-120-disk-0,iothread=1,size=8G

# Déplacement vers zfs-tank
qm move-disk 120 scsi0 zfs-tank --delete 1

# Vérification après déplacement
qm config 120 | grep scsi0
```

```
copying volume 'local-lvm:vm-120-disk-0' from current storage 'local-lvm' to target
storage 'zfs-tank'
transferred 8.0 GiB of 8.0 GiB (100.00%)
removing source volume 'local-lvm:vm-120-disk-0' after successful copy
  Logical volume "vm-120-disk-0" successfully removed.

scsi0: zfs-tank:vm-120-disk-0,iothread=1,size=8G
```

Partie 13 — Sauvegarde et restauration

Backup de la VM 120 vers nfs-backup

```
vzdump 120 --storage nfs-backup --mode snapshot --compress zstd
```

```
INFO: backup mode: snapshot
INFO: issuing guest-agent 'fs-freeze' command
INFO: issuing guest-agent 'fs-thaw' command
INFO: backup is sparse: 5.58 GiB (69%) total zero data
INFO: transferred 8.00 GiB in 10 seconds (819.2 MiB/s)
INFO: archive file size: 1.12GB
INFO: Backup job finished successfully
```

Résultat remarquable

8 Go de disque → 1.12 Go de backup (compression 86%). Combinaison sparse detection (69% de zéros détectés) + compression zstd. Le sparse ZVOL ne stockant que les blocs écrits, vzdump ne sauvegarde que les données réelles.

Restauration sous VM 121 sur local-lvm

```
qmrestore nfs-backup:backup/vzdump-qemu-120-2026_06_09-21_40_07.vma.zst 121 \
  --storage local-lvm \
  --unique 1

qm start 121
# IP obtenue : 192.168.131.149
curl http://192.168.131.149
```

```
total bytes read 8589934592, sparse bytes 5995032576 (69.8%)
space reduction due to 4K zero blocks 2.82%

zfs-test01 operationnel
```

Architecture 3 stockages

Backup source : ZFS (zfs-tank) → Fichier backup stocké sur : NFS (nfs-backup) → VM restaurée sur : LVM-thin (local-lvm). Trois technologies impliquées, c'est exactement le scénario de production recommandé.

Validation finale — Scénario complet

```
echo "=== PARTIE 1 - Stockages Proxmox ===" && pvesm status
echo "=== PARTIE 2 - NFS ===" && showmount -e 192.168.131.147
echo "=== PARTIE 3 - iSCSI ===" && pvesm list iscsi-store
echo "=== PARTIE 4 - LVM-thin ===" && lvs pve/data
echo "=== PARTIE 5 - Pool ZFS ===" && zpool status tank | grep -A8 config
echo "=== PARTIE 6 - Datasets ===" && zfs list tank/backups tank/iso
echo "=== PARTIE 7 - ZVOL vs Dataset ===" && zfs list | grep -E "backups|vm-120"
echo "=== PARTIE 8 - Compression ===" && zfs get compression,compressratio tank
echo "=== PARTIE 9 - Scrub ===" && zpool status tank | grep scan
echo "=== PARTIE 10 - Mirror vs Simple ===" && zpool list
echo "=== PARTIE 11 - Service VM 120 ===" && curl http://192.168.131.148
echo "=== PARTIE 12 - Disque sur ZFS ===" && qm config 120 | grep scsi0
echo "=== PARTIE 13 - Backup ===" && pvesm list nfs-backup
echo "=== PARTIE 14 - Restauration ===" && curl http://192.168.131.149
```

```
=== PARTIE 1 - Stockages Proxmox ===
iscsi-store   iscsi    active         0          0          0    0.00%
local          dir     active   61349692   37732912   20467980   61.50%
local-lvm  lvmthin     active  118427648   27546270   90881377   23.26%
nfs-backup     nfs     active    7669824    5757120    1501568   75.06%
zfs-tank   zfspool     active    9650176    8522023    1128153   88.31%
=== PARTIE 2 - NFS ===
Export list for 192.168.131.147:
/srv/nfs/proxmox 192.168.131.0/24
=== PARTIE 3 - iSCSI ===
iscsi-store:0.0.0.scsi-36001405b1ca6228095d46d78240548e6 raw  images  3221225472
=== PARTIE 4 - LVM-thin ===
  data pve twi-aotz-- 112.94g   23.26  2.08
=== PARTIE 5 - Pool ZFS ===
        NAME        STATE     READ WRITE CKSUM
        tank        ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            loop1   ONLINE       0     0     0
            loop2   ONLINE       0     0     0
=== PARTIE 6 - Datasets ===
tank/backups    24K  1.08G    24K  /tank/backups
tank/iso        24K  1.08G    24K  /tank/iso
=== PARTIE 7 - ZVOL vs Dataset ===
tank/backups          24K  1.08G    24K  /tank/backups
tank/vm-120-disk-0  8.13G  7.69G  1.51G  -
=== PARTIE 8 - Compression ===
tank  compression    lz4     local
tank  compressratio  1.61x   -
=== PARTIE 9 - Scrub ===
  scan: scrub repaired 0B in 00:00:02 with 0 errors on Tue Jun  9 21:10:29 2026
=== PARTIE 10 - Mirror vs Simple ===
demo-simple  9.50G   123K  9.50G  ...  ONLINE
tank         9.50G  1.51G  7.99G  ...  ONLINE
=== PARTIE 11 - Service VM 120 ===
zfs-test01 operationnel
=== PARTIE 12 - Disque sur ZFS ===
scsi0: zfs-tank:vm-120-disk-0,iothread=1,size=8G
=== PARTIE 13 - Backup ===
nfs-backup:backup/vzdump-qemu-120-2026_06_09-21_40_07.vma.zst  vma.zst  backup
1204959152
=== PARTIE 14 - Restauration ===
zfs-test01 operationnel
```

Synthèse — Points clés à retenir

La distinction fondamentale

La distinction fondamentale est fichier vs bloc : NFS exporte un répertoire accessible en réseau comme un dossier monté, tandis qu'iSCSI expose un périphérique bloc (LUN/disque virtuel) que le client partitionne et formate selon ses besoins. ZFS unifie ces deux approches avec les datasets (stockage fichier, montés directement dans l'arborescence) et les ZVOLs (stockage bloc, visibles comme des périphériques de type disque avec un mountpoint à "-").

Thin provisioning

LVM-thin et ZFS sparse reposent sur la même logique de thin provisioning : présenter logiquement plus d'espace qu'on n'en consomme physiquement au départ, LVM-thin l'implémentant au niveau du Volume Group tandis que ZFS l'intègre nativement dans le pool.

Intégrité ZFS

ZFS se distingue par ses checksums de bout en bout associés à chaque bloc écrit ; le scrub les vérifie périodiquement et peut corriger automatiquement les corruptions silencieuses lorsqu'une redondance est disponible, ce que des systèmes de fichiers classiques comme ext4 ou XFS ne réalisent pas nativement.

Élément	État	Preuve
Stockages identifiés	☑	pvesm status — 5 types actifs
NFS	☑	showmount + nfs-backup actif
iSCSI	☑	LUN raw visible dans Proxmox
LVM-thin	☑	lvs pve/data — Data% 23.26%
Pool ZFS mirror	☑	zpool status — mirror-0 ONLINE
Dataset ZFS	☑	tank/backups, tank/iso montés
ZVOL	☑	tank/vm-120-disk-0 — MOUNTPOINT -
Compression lz4	☑	compressratio 1.61x
Scrub ZFS	☑	0 errors, 0B repaired
Mirror vs RAIDZ	☑	demo-simple vs tank comparés
VM sur stockage précis	☑	nginx zfs-test01 operationnel
Déplacement de disque	☑	local-lvm → zfs-tank documenté

Backup	☑	8 Go → 1.12 Go sur nfs-backup
Restauration	☑	VM 121 sur local-lvm — service OK